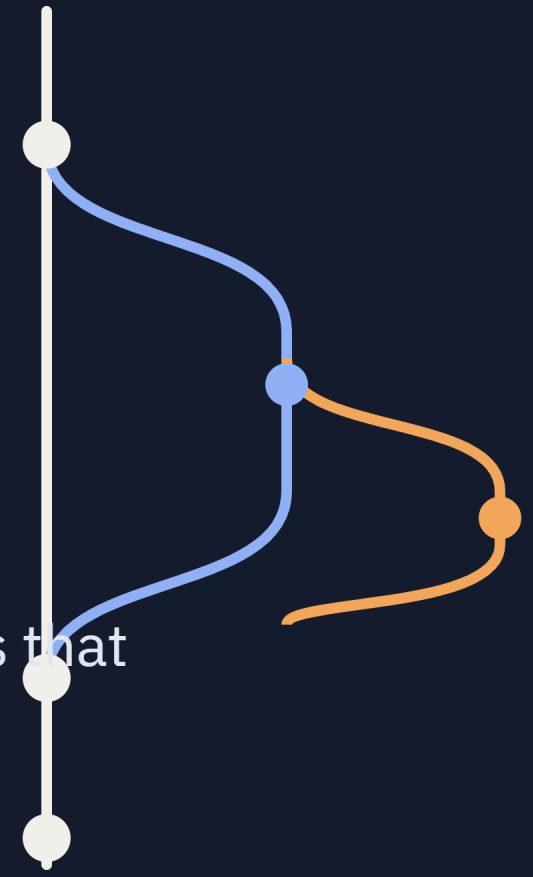
GIT & GITHUB · TEAM WORKFLOW

GitHub Branching Techniques

Five strategies, naming rules, merge options and the settings that keep main safe.



WHY IT MATTERS

A branching strategy is a team agreement

Isolate work

Each feature or fix lives on its own branch, so main stays stable while work is in progress.

Review before merge

Pull requests add code review, automated CI checks and a place to discuss changes.

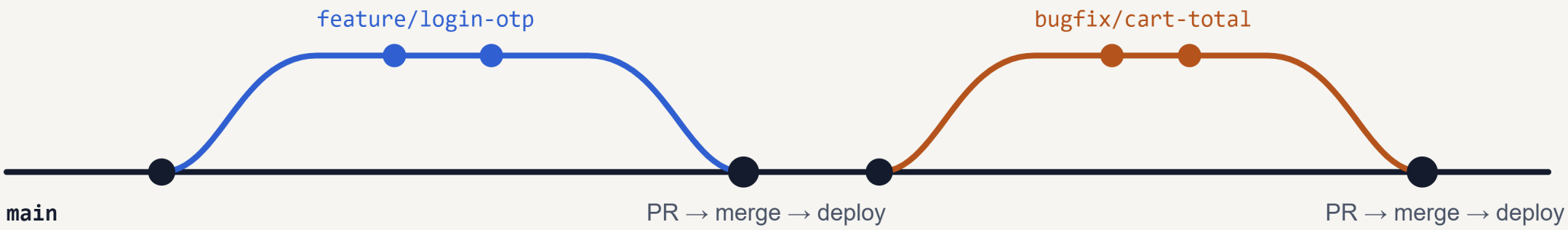
Ship predictably

A shared model tells everyone where code goes, how releases happen and how hotfixes reach production.

The core loop: **branch** → **commit** → **pull request** → **review** → **merge** → **deploy**

STRATEGY 1 · SIMPLEST AND MOST POPULAR

GitHub Flow



How it works

Branch from main, commit, open a pull request, get a review, merge and deploy.

Best for

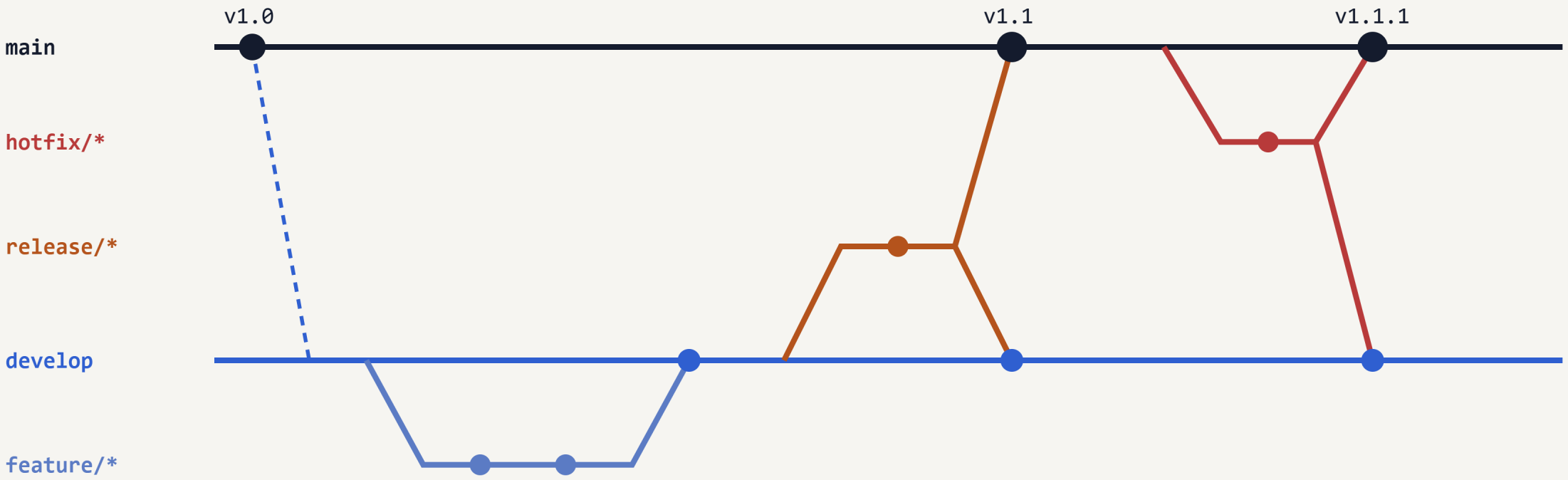
Web apps with continuous deployment and small teams, like most Laravel and WordPress projects.

Watch out

main must always be deployable, so CI checks and reviews are essential.

STRATEGY 2 · STRUCTURED RELEASES

Git Flow: two long-lived branches



STRATEGY 2 · GIT FLOW

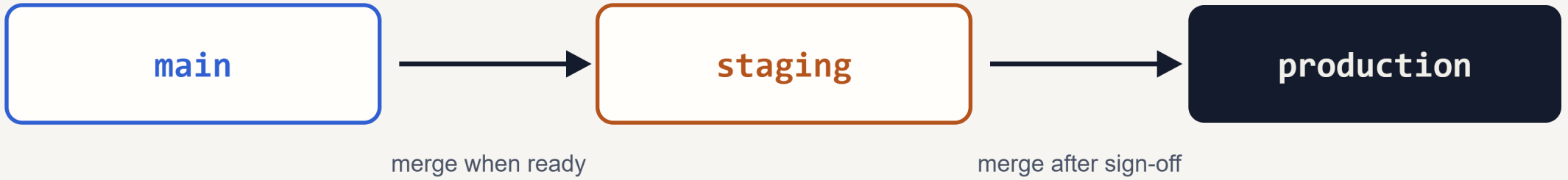
Each branch has one job

Branch	Starts from	Merges into	Purpose
main	—	—	Production-ready code, tagged per release
develop	main	release/*	Integration branch for the next release
feature/*	develop	develop	One feature per branch
release/*	develop	main + develop	Stabilise, test and bump the version
hotfix/*	main	main + develop	Urgent fixes to production

Best for versioned products with planned releases: mobile apps, packages, on-premise software. **Trade-off:** heavy overhead for teams that deploy daily.

STRATEGY 3 · GITLAB FLOW

Environment branches



How it works

Code only moves forward: main → staging → production, each step by merge or pull request.

Best for

Projects that need QA or client approval on a staging server before going live.

Rule

Fix bugs on main first, then promote. Never commit directly to production.

STRATEGY 4 · FASTEST DELIVERY

Trunk-Based Development

Everyone integrates into main at least once a day. Unfinished work ships switched off, behind feature flags.

Tiny branches

Branches live hours, not weeks, so merge conflicts stay small.

Feature flags

Deploy code turned off, then switch features on when they’re ready.

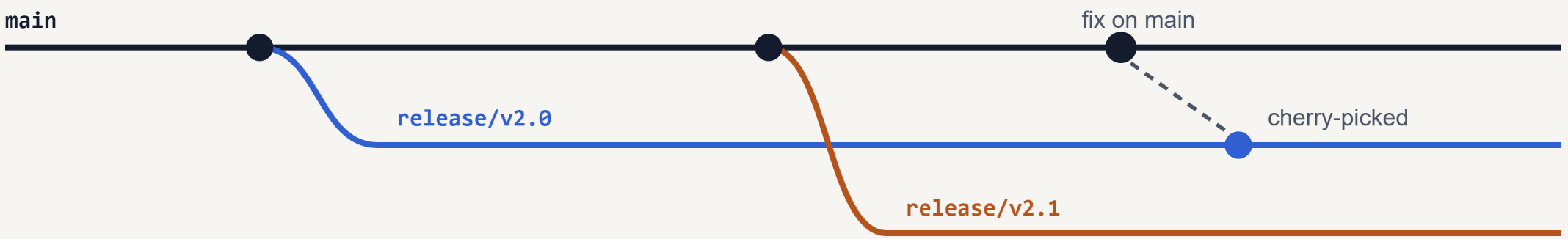
Strong CI

Automated tests on every push are what make this safe.

Best for experienced teams that deploy many times a day.

STRATEGY 5 · MULTIPLE SUPPORTED VERSIONS

Release branches



How it works

Cut **release/v2.1** from main when it’s ready; only fixes go into it afterwards.

Best for

SaaS or libraries that support several versions at once.

Tip

Fix on main first, then cherry-pick into each supported release.

SIDE BY SIDE

Comparing the five strategies

Strategy	Long-lived branches	Release style	Process	Best for
GitHub Flow	main	Continuous	Low	Web apps, small teams
Git Flow	main, develop	Scheduled versions	High	Mobile apps, packages
GitLab Flow	main, staging, production	Promoted per environment	Medium	Projects with client sign-off
Trunk-Based	main	Continuous + flags	Low, but needs strong CI	Mature teams
Release branches	main, release/*	Versioned, supported	Medium	Multi-version SaaS, libraries

CONVENTIONS

Branch naming

```
feature/user-auth
bugfix/cart-total-rounding
hotfix/payment-gateway-timeout
release/v1.4.0
chore/upgrade-laravel-11
feature/YAL-123-export-pdf
```

- Start with the type: feature, bugfix, hotfix, release or chore
- Lowercase words joined with hyphens
- Add the ticket ID when there is one
- Short and descriptive
- Delete the branch after it’s merged

CONVENTIONS

Merge strategies on GitHub

Strategy	What you get	Use when
Merge commit	Full branch history plus a merge commit	You want a complete audit trail
Squash & merge	One clean commit per pull request	Feature branches with messy commits (most common)
Rebase & merge	Linear history, every commit kept	Commits are already clean and meaningful

Turn on only the options your team agrees on in the repository’s settings, under Pull Requests.

GITHUB SETTINGS

Keep main safe

Branch protection / rulesets

Require pull requests, approvals and passing checks; block force-pushes.

CODEOWNERS

Automatically request reviews from the owners of changed files.

GitHub Actions

Run tests and linting on every pull request.

Auto-delete branches

Remove head branches automatically after merging.

Draft pull requests

Share work in progress early without risk of an accidental merge.

Linear history

Optionally require squash or rebase so history stays clean.

IN PRACTICE

A day in GitHub Flow

```
# 1. Start from an up-to-date main
git checkout main && git pull
# 2. Create a feature branch
git checkout -b feature/invoice-pdf
# 3. Commit your work
git add . && git commit -m "Add invoice PDF export"
# 4. Push, then open a pull request on GitHub
git push -u origin feature/invoice-pdf
# 5. After squash & merge, clean up
git checkout main && git pull && git branch -d feature/invoice-pdf
```

RECOMMENDATION

Which strategy should you use?

Client websites and Laravel apps	**GitHub Flow, plus a staging branch for client sign-off**
Versioned products and mobile apps	**Git Flow or release branches**
Mature teams with strong CI	**Trunk-based development with feature flags**

WRAP-UP

Key takeaways

01 Keep branches short-lived: small pull requests merge faster and conflict less.

02 Protect main with required reviews and passing CI checks.

03 Agree on branch naming and one merge strategy as a team.

04 Pick the simplest strategy that fits how you actually release.